

PowerUp Practice Project — Course Explorer

Why this project exists

On Saturday we covered four sessions in about three hours: React Router, Context, Redux Toolkit, and Redux with more than one slice, selectors and `localStorage`. That was fast on purpose — you saw every main idea once.

This project is where you practise all of them **yourself**, in one small app, at your own speed. Nothing here is new. Every tool you need was shown on Saturday and is explained in detail in the `docs/session-01` ... `docs/session-04` PDFs in this repository.

Plan for several sittings. Most students need about **8–12 hours** in total. A good rhythm is 3–4 sittings of 2–3 hours (see "Suggested plan" at the end).

The app in one sentence

Course Explorer is a small catalog of courses. You can browse all courses, open one course, save the ones you like, see what you opened recently, and switch between a light and a dark theme.

What you already have

Everything is inside the `course-explorer/` folder:

File	What it gives you
<code>src/data/courses.ts</code>	11 courses, ready to import. No server, no fetch.
<code>src/types.ts</code>	The <code>Course</code> type.
<code>src/index.css</code>	A few starter styles. Colors are CSS variables — change anything you like.
<code>src/App.tsx</code> , <code>src/main.tsx</code>	A placeholder screen. You will replace it.
<code>package.json</code>	React, React Router, Redux Toolkit and React Redux are already installed.

One course looks like this:

TS

```
{
  id: 1,
  title: "React Fundamentals",
  category: "Frontend",
  instructor: "Sara",
  duration: "6 hours",
  level: "Beginner",
  description: "Components, props, state and events. ...",
  featured: true,
}
```

Start the project (from the repository root):

BASH

```
cd course-explorer
npm install
npm run dev
```

Open the address Vite prints (normally `http://localhost:5174`).

Screens and routes

TEXT

Navbar (always):

Home Courses Saved (2) [Dark mode]

/	Home page – welcome, featured courses, saved count, recently viewed
/courses	All courses – one card per course
/courses/:id	One course – full details + Save / Unsave
/saved	Saved courses only
*	Anything else – "404 – Page not found"

How the screens connect:

TEXT

Home	→	/courses/:id	(featured card "Details", or a recently-viewed link)
/courses	→	/courses/:id	(card "Details" link)
/saved	→	/courses/:id	(card "Details" link)
/courses/:id	→	back to where you came from	

The two kinds of state — and which tool holds each one

This is the most important idea of the project. **Context and Redux are different tools.** Each piece of state lives in exactly one place.

State	Where it lives	Why there
Theme ("light" or "dark")	Context — ThemeContext	Simple app-wide UI setting. Many components read it, it changes rarely, there is no logic around it.
Saved course ids	Redux — saved slice	Shared app data. Changed from several screens, read by several screens, needs clear actions.
Recently viewed course ids	Redux — recentlyViewed slice	Shared app data with rules (no duplicates, newest first, maximum 5).
Selected category filter on /courses	Local <code>useState</code> in the page	Only one page cares about it.

Do not put the theme in Redux, and **do not** put saved or recently viewed courses in Context. If you notice the same data in two places, stop and fix it — that is exactly the mistake this project trains you to avoid.

Redux state you should end up with

TEXT

```
store
├── saved
│   └── savedIds: number[]      e.g. [7, 1]
└── recentlyViewed
    └── ids: number[]           e.g. [7, 6, 5, 4, 2] (newest first, max 5)
```

Store **ids only** in Redux, not whole course objects. The course details already live in `courses.ts`; a selector can turn ids into courses when a component needs them.

Difficulty levels

The work is split into three levels. Build them **in order** — each level uses what the level before it built.

- **CORE — required for everyone.**
- **STRETCH — expected.** Strongly recommended for everyone, required if you want to say the project is "complete".
- **CHALLENGE — for students who want to go further.** Do it after STRETCH is fully working.

CORE (required)

C1. Routing

1. Wrap the app in `BrowserRouter`.
2. Use `Routes` and `Route` for `/`, `/courses`, `/courses/:id`, `/saved` and `*`. In CORE, `/saved` may show only a heading — you will build it in STRETCH.
3. Create one component per page inside a `src/pages/` folder.
4. Build a Navbar with `NavLink` for Home, Courses and Saved. The current page's link must look different (active style). Make sure **Home** is not active on every page.
5. Add a 404 page for unknown URLs, with a `Link` back to Home.

C2. Course list — `/courses`

1. Show one card per course. Each card shows at least: title, **category**, instructor, level.
2. Each card has a **Details** link to `/courses/:id` (use `Link`).
3. Each card has a **Save / Unsave** button (see C5).

Tip: build a reusable `CourseCard` component — you will use it on three pages.

C3. Course details — `/courses/:id`

1. Read the id from the URL with `useParams()`.

2. Find the matching course in `courses.ts` and show all its information.
3. If no course has that id (try `/courses/999`), show a clear "Course not found" message with a link back to `/courses`. The page must not crash.
4. Add a **Save / Unsave** button.
5. Add a **Back** button that returns to the previous page (`useNavigate`).

Remember: values from `useParams()` are **strings**, and the ids in `courses.ts` are **numbers**.

C4. Theme with Context

1. Create `src/context/ThemeContext.tsx` with `createContext`, a `ThemeProvider` component holding the theme in `useState`, and a custom hook `useTheme()` that reads the context and throws a clear error if it is used outside the provider — the same shape as `NotificationContext` in B4F Hub.
2. The context gives at least the current `theme` and a function to switch it.
3. Put a theme button in the Navbar.
4. The whole page really changes color in dark mode (background, text, cards, Navbar). The CSS variables in `index.css` make this easier — add a dark set of values.

C5. Redux — the `saved` slice

1. Create `src/store/savedSlice.ts` with `createSlice`. `State: savedIds: number[]`.
2. Actions: a `toggleSaved(id)` action (or a `save` and an `unsave` action) and a `clearSaved` action.
3. Create the store with `configureStore` in `src/store/store.ts`, and export the `RootState` and `AppDispatch` types.
4. Create typed hooks `useAppDispatch` and `useAppSelector` in `src/store/hooks.ts`, and use them everywhere instead of the plain hooks.
5. Wrap the app in the React Redux `Provider`.
6. Save / Unsave works on the course cards **and** on the details page, and both always agree.
7. The Navbar shows the number of saved courses (for example `Saved (2)`), and it updates immediately.
8. Put small selectors next to the slice: `selectSavedIds`, `selectSavedCount`, and `selectIsSaved(state, id)`.

C6. Persist saved courses with `localStorage`

Session 4 made saved items survive a reload. Do the same here:

1. A storage-key constant, for example `SAVED_COURSE_IDS_KEY = "savedCourseIds"`.
2. A loader function: read the key; if nothing is stored, or `JSON.parse` throws, return `[]`.
3. Use the loader for the slice's `initialState`.
4. In `store.ts`, use **one** `store.subscribe(...)` callback that writes `savedIds` to `localStorage` after every change.

Test: save two courses → reload the page → both are still saved, and the Navbar count is still right.

Remember: Redux itself keeps state **in memory only**. The persistence is your own code around Redux, not something Redux does.

STRETCH (expected)

S1. Redux — the `recentlyViewed` slice

1. Create `src/store/recentlyViewedSlice.ts`. State: `ids: number[]`.
2. Action `recordView(id)`:
 - the newest course goes **first**;
 - **no duplicates** — opening the same course again moves it to the front;
 - keep at most **5** ids.
3. Action `clearRecentlyViewed`.
4. Add the slice to the store, next to `saved`.
5. Opening `/courses/:id` for an existing course dispatches `recordView`. A course that does not exist is never recorded.

S2. Selectors

1. Create `src/store/selectors.ts` for selectors that need course data or more than one slice.
2. Write at least:
 - `selectSavedCourses(state)` — the saved ids turned into full `Course` objects;
 - `selectRecentlyViewedCourses(state)` — the same for recently viewed, in the same order.
3. Selectors should **reuse** the small selectors from the slices (for example `selectSavedIds`) instead of reading `state.saved.savedIds` again by hand.
4. Components call these selectors. **Do not** repeat the same filtering code inside several components.

S3. The Saved page — `/saved`

1. Show only the saved courses, using `selectSavedCourses` and your `CourseCard`.
2. If nothing is saved, show a friendly message and a link to `/courses`.
3. Add a "Clear saved" button.
4. Unsaving a course on this page removes it from the list at once.

S4. The Home page — `/`

1. A welcome section.
2. **Featured courses** — the courses with `featured: true`, shown with `CourseCard`.
3. A **saved summary** — how many courses are saved, with a link to `/saved`.
4. A **Recently viewed** list — course titles that link to their details page, newest first. If it is empty, say so.

CHALLENGE (go further)

X1. Persist recently viewed too

1. A **second** storage key with its own name, for example `RECENTLY_VIEWED_IDS_KEY = "recentlyViewedCourseIds"`. It must be a different string from the saved key.
2. A loader with the same try/catch-to-`[]` fallback, used for the slice's `initialState`.

3. **Extend** the existing `store.subscribe` callback to also write `recentlyViewed.ids`. Do **not** add a second `store.subscribe` call.

Test: open three courses → reload → the same recently viewed list, in the same order. Clear it → reload → it stays empty. Saved courses still work exactly as before.

X2. Filter by category on `/courses`

1. Show one button (or a `<select>`) per category, plus **All**.
2. Clicking a category shows only those courses. **All** shows every course again.
3. Build the category list from the data — do not type the category names by hand.
4. The selected category is **local** state of the page (`useState`). It is not Redux and not Context: no other screen needs it.

X3. Recently viewed but not saved yet

1. Add `selectRecentlyViewedNotSaved(state)` to `selectors.ts`: the recently viewed courses whose id is **not** in the saved ids. Reuse your other selectors.
2. On the Home page, show a short note such as "3 of your recently viewed courses are not saved yet."
3. The note never shows the number 0 — it disappears when every recently viewed course is saved.
4. It updates immediately when you save or unsave a course.

Rules — what to use and what not to use

Use everything from Sessions 1–4 and PowerUp: components, props, `useState`, `useEffect`, `BrowserRouter`, `Routes`, `Route`, `Link`, `NavLink` (with `end`), `useParams`, `useNavigate`, `createContext`, a `Provider`, `useContext` inside a custom hook, `configureStore`, `createSlice`, `Provider`, `useAppDispatch` / `useAppSelector`, plain selector functions, `localStorage`, `JSON.stringify` / `JSON.parse`, `store.subscribe`, and array methods like `.filter()`, `.find()`, `.forEach()`, `.includes()`, `.map()`, `.slice()`.

Do not use (not covered yet): `Redux Persist` (the library), `createSelector` / `reselect`, `createAsyncThunk` or any thunk, custom middleware, `extraReducers`, `RTK Query`, `React Query`, `createEntityAdapter`, `useReducer`, `useSearchParams`, nested routes or layout routes, a backend or API, `fetch`, authentication, `Next.js`.

Do not change the `client/` or `server/` folders. They are the B4F Hub code from Session 4 — read them as an example, but do all your work inside `course-explorer/`.

A console warning you may see

When a selector builds a new array (for example `selectSavedCourses`), `React Redux` may print a **yellow warning** in the browser console: *"Selector ... returned a different result when called with the same parameters."* This is a development-only performance hint, not an error, and the app still works correctly. You can ignore it in this project. (The tool that removes it, `createSelector`, is not covered yet.)

Test your app

Go through every line. Each one should be true.

#	Do this	Expected
1	Click Home, Courses, Saved in the Navbar	The URL changes, the right page shows, the right link is active
2	Open <code>/courses</code>	One card per course, each with category, Details and Save
3	Click Details on a card	<code>/courses/<id></code> shows that course's full information
4	Open <code>/courses/999</code>	"Course not found" with a link back — no crash
5	Open <code>/something-wrong</code>	Your 404 page
6	Save a course on its card, then open its details	The details page shows Unsave
7	Save two courses, reload	Both still saved, Navbar count still right
8	Switch to dark mode, visit every page	Every page is dark
9	Open 7 different courses, go Home	5 courses in Recently viewed, the last one opened first
10	Open the same course twice	It appears only once, at the front
11	Open <code>/saved</code> , click Unsave on a card	The card disappears at once
12	(CHALLENGE) Open three courses, reload	Same recently viewed list, same order
13	(CHALLENGE) Choose a category, then All	Only that category, then every course again
14	(CHALLENGE) Clear everything, open 3 courses, save 1 of them	Home says 2 are not saved yet

Also run, inside `course-explorer/` :

BASH

```
npm run build
npm run lint
```

Both must finish with **0 errors**.

Explain it in your own words

Create `course-explorer/NOTES.md` and answer in 3–6 sentences each:

1. Why is the theme in Context, but saved courses in Redux?

2. What happens to Redux state when you reload the page, and what exactly did **your** code do to bring saved courses back?
3. Why do we store course **ids** in Redux instead of whole course objects?
4. Pick one selector you wrote. Why is it better to have it in `selectors.ts` than to write the same filter inside the components?

Suggested file structure

You may organise files differently if you can explain why.

TEXT

```
course-explorer/src/
├─ main.tsx
├─ App.tsx
├─ types.ts
├─ index.css
├─ data/courses.ts
├─ context/ThemeContext.tsx
├─ store/
│   ├─ store.ts
│   ├─ hooks.ts
│   ├─ savedSlice.ts
│   ├─ recentlyViewedSlice.ts
│   └─ selectors.ts
├─ components/
│   ├─ Navbar.tsx
│   └─ CourseCard.tsx
└─ pages/
    ├─ HomePage.tsx
    ├─ CoursesPage.tsx
    ├─ CourseDetailPage.tsx
    ├─ SavedPage.tsx
    └─ NotFoundPage.tsx
```

Suggested plan

#	Goal	Review first
1	C1–C3: all routes, Navbar, course list, details page, 404	docs/session-01
2	C4–C6: theme Context, <code>saved slice</code> , Save / Unsave, persistence	docs/session-02 , docs/session-03 , docs/session-04
3	S1–S4: <code>recentlyViewed slice</code> , selectors, Saved page, Home page	docs/session-03 , docs/session-04
4	CHALLENGE, the test table, <code>NOTES.md</code> , build and lint	docs/session-04

Commit at the end of every sitting, with a message that says what you finished, for example `Course Explorer: routes and course list`.

Submitting

1. Your instructor will create your own branch, named `student-<your-github-username>` in lowercase, and tell you when it is ready. Work only on that branch.
2. Commit your work often and push to **your own branch**. Never push to `main`.
3. Your branch must contain: the working app inside `course-explorer/`, and `course-explorer/NOTES.md` with your answers.
4. When you finish, tell your instructor which levels you completed (CORE, STRETCH, CHALLENGE).

Done when

- Every CORE item works, and test lines 1–11 pass.
- STRETCH is complete for a "complete" project.
- The theme lives only in Context; saved and recently viewed courses live only in Redux.
- Saved courses survive a reload.
- `npm run build` and `npm run lint` finish with 0 errors.
- `NOTES.md` answers all four questions in your own words.